NODE.JS • DAY 22

Practical process Usage

DAY 22 COMPLETE

What you learned

Day 22 focused on the Node.js process object and a few practical features used from the command line and inside backend programs.

Think of process as information and controls related to the Node.js program that is currently running.

1. process.argv — Command-line arguments

argv means argument vector. It is an array containing the arguments passed to your Node.js program from the command line.

```
node app.js Bhavana 26

process.argv[2] // "Bhavana"
process.argv[3] // "26"
```

The user-supplied arguments begin at index 2. Node uses the earlier positions for the Node executable and the script path.

Important: command-line arguments are strings.

```
const age = process.argv[3];

typeof age;
// "string"

const numericAge = Number(process.argv[3]);
// 26
```

2. Validating command-line input

Before using required input, we can check whether it was provided.

```
const name = process.argv[2];

if (!name) {
console.log("Please provide a name");
process.exit();
}

console.log(`Hello ${name}`);
```

If no name is provided, process.argv[2] is undefined. Since undefined is falsy, !name becomes true.

3. process.exit() — Terminate the current process

process.exit() stops the currently running Node.js process.

```
console.log("Before");

process.exit();

console.log("After");
```

Output:

```
Before
```

The After line does not execute because the process has already terminated.

4. process.cwd() — Current Working Directory

cwd means Current Working Directory. It represents the directory from which the Node.js command is currently being run.

```
console.log(process.cwd());
```

For example, if your terminal is currently inside shop/, then process.cwd() points to shop/, even if the JavaScript file you execute is inside a subfolder.

5. __dirname vs process.cwd()

These two are related to paths, but they answer different questions.

Feature	Meaning
__dirname	Where the current JavaScript file is located.
process.cwd()	The directory you are currently running the Node command from.

Example

```
shop/
├── app.js
└── utils/
└── helper.js

# terminal is inside shop/
node utils/helper.js

__dirname → shop/utils
process.cwd() → shop
```

Easy way to remember

__dirname	“Where is this file?”
process.cwd()	“Where am I running the command from?”

6. Mini Challenge

You created a command-line program that accepts a name and age, validates both values, converts age to a number, and prints the result.

6. Mini Challenge — Your Solution

```
const name = process.argv[2];
const age = Number(process.argv[3]);

if (!name || !age) {
console.log('Please provide a name and age');
process.exit();
}

console.log(`Name: ${name}`);
console.log(`Age: ${age}`);

// run:
// node exercise.js Bhavana 26
```

Expected output:

```
Name: Bhavana
Age: 26
```

□ Important logic point

The original condition used &&, which would only show the error when both values were missing.

The corrected condition is:

```
if (!name || !age) {
```

Using || means the program rejects the input when either required value is missing.

Example

```
node exercise.js Bhavana
→ age is missing
→ validation fails

node exercise.js
→ name and age are missing
→ validation fails

node exercise.js Bhavana 26
→ both values are present
→ program continues
```

Day 22 Quick Reference

Remember	Meaning
process.argv	Gets command-line arguments.
process.argv[2]	First user-supplied argument in a typical Node command.
Number(...)	Converts a string such as "26" into the number 26.
process.exit()	Terminates the current Node.js process.
process.cwd()	Gets the current working directory.
__dirname	Gets the directory containing the current JavaScript file.

Day 22 Assessment

Final check: 3/3 correct.

- process.argv → command-line arguments
- process.cwd() → current working directory
- process.exit() → terminate the current process

Practical challenge: completed successfully.

DAY 22 — COMPLETE

Next → NPM / Packages

We'll continue the remaining Node.js foundation before moving to Express.